Keyword	Definition
Variable	A named value in which data can be stored by a computer program. The data can be changed as the program runs.
Constant	A named value which is accessed by a computer program but is fixed and cannot be changed.
Operator	Operators compare or modify values in computer code. Examples include + < and NOT.
Input	Data or information which is put into the program. This could be entered via a user or taken from a sensor.
Output	Data or information which the program produces. This could be shown on a screen, sent to a printer, or saved to a file.
Assignment	The process of placing a value into a variable.
Sequence	Following through a set of steps in a particular order.
Selection	Taking different paths through the code depending on a condition.
Iteration	Repeating a set of steps several times.
Count Controlled Loop	Using a counter to loop through one or more lines of code until they have been run a certain number of times.
Condition Controlled Loop	Using a loop to repeat one or more lines of code until a certain condition is met.
Arithmetic Operator	An operator which uses maths to modify numbers such as +
Boolean Operator	An operator which uses a logical criteria to compare two values such as < or AND
Integer	A whole number without a decimal point or fraction. E.g. 1, 100 or 88122
Real	A number which may include a decimal point such as 11.2
Boolean	A value which can be only TRUE or FALSE.
Character	A single symbol, for example a letter, number or punctuation mark.
String	A number of characters together, usually to form a word or sentence.
Casting	The process of forcing a value into a certain format, for example forcing a telephone number to be stored as a string and not an integer.
String Manipulation	A broad term for a number of ways to change or modify strings.
Concatenation	The process of joining two or more strings together.
Slicing	The process of splitting a string into two or more parts.
File Handling	Ways in which computer code can interact with files on the computer's disk, such as opening them or writing data to them.
Record	A single row or related information in a database table
Table	A group of records and fields stored together in a database.

Field	A part of a record in the database which stores specific information such as a name or address.
SQL	Structured Query Language. A programming language designed to be used with databases.
Array	A set of data items stored together with a single name and accessed by a program.
Sub Program	A smaller part of the program which performs a specific task.
Function	A sub program which performs a specific task and always returns a specific value when run.
Procedure	A sub program which performs a specific task but does not always return a value.
Random Number	A number generate by computer at random.

2.2.1 Programming Fundamentals

2.2.1 Programming fundamentals

- ☐ The use of variables, constants, operators, inputs, outputs and assignments
- ☐ The use of the three basic programming constructs used to control the flow of a program:
  - Sequence
  - Selection
  - Iteration (count- and condition-controlled loops)
- ☐ The common arithmetic operators
- ☐ The common Boolean operators AND, OR and NOT

Required

- ✓ Practical use of the techniques in a high-level language within the classroom
- ✓ Understanding of each technique
- ✓ Recognise and use the following operators:

Comparison operators		Arithmetic operators	
==	Equal to	+	Addition
!=	Not equal to	–	Subtraction
<	Less than	*	Multiplication
<=	Less than or equal to	/	Division
>	Greater than	MOD	Modulo
>=	Greater than or equal to	DIV	Quotient
		^	Exponentiation (to the power)

Concept	Meaning (student-friendly)	Code Example
Variable	A temporary value that can change during a program. Like a box holding data.	`score = 0` `lives = 3`
Updating a variable	Reassigning a new value to the same variable.	`score = score + 1` This is known as increment
Using in expressions	Variables can be used in calculations.	`myNumber = 5` `print(myNumber * 3)`
Constant	A value that must **not change** once set.	`const pi = 3.14`
Constant rule	Constants **cannot be reassigned**.	`pi = 4` ❌
Copying a constant	A variable copied from a constant can still change.	`my_pi = pi` `my_pi = 4`
Input	Gets data from the user (as a string).	`name = input("Enter name")`
Output	Displays data to the user.	`print(name)`
Input + output	Common exam pattern	`age = input("Age?")` `print(age)`

Operators

Operator type	What it is used for	Example (OCR style)	Operators	Common exam mistakes
Arithmetic operators	Perform calculations on numbers (maths).	total = 7 + 3	+ – * / DIV MOD ^	Using / instead of DIV when an integer answer is required; forgetting MOD gives the remainder; mixing strings and numbers without conversion.
Relational (comparison) operators	Compare values and return TRUE or FALSE.	age >= 18	== != < > <= >=	Using = instead of ==; writing assignment instead of comparison; forgetting comparisons return TRUE/FALSE, not numbers.
Boolean (logical) operators	Combine or change TRUE / FALSE values.	hasTicket AND isAdult	AND OR NOT	Using AND when OR is needed; forgetting NOT reverses the condition; missing brackets when combining conditions.

Arithmetic → does the maths
Relational → asks a question
Boolean → combines answers

Iteration (Loops) – OCR Exam Reference Language



Summary Table

Loop type	What it does	Key feature	ERL example

FOR (count-controlled)	Repeats code a set number of times	Runs from a start value to an end value (inclusive)	`for i = 1 to 10`
			`  print(i)`
			`  next i`

FOR with step	Repeats code using a **step value**	Can count up or down	`for i = 0 to 10 step 2` `  print(i)`

FOR countdown	Counts backwards	Uses a negative step	`for i = 5 to 0 step -1`
			`  print(i)`
			`  print("Blastoff!")`

WHILE (condition-controlled)	Repeats while a condition is **true**	May run zero times	`x = ""`
			`while x != "parrot"`
			`  x = input("password : ")`
			`  print "you guessed", xx)`
			`endwhile`
			`  print "correct!")`
			`print("correct!")`

DO ... UNTIL (condition-controlled)	Repeats until a condition is **true**	Always runs at least once	

Key exam differences to remember

- ✓ **FOR** loops → use when you know **how many times**
- ✓ **WHILE** loops → check condition **before** running
- ✓ **DO...UNTIL** loops → check condition **after** running (runs once minimum)
- ✓ FOR loop bounds are **inclusive**
- ✗ **Do not change the loop variable** ('*i*') inside a FOR loop ✗

Practice coding using the examples below.
Variables Operators Selection Iteration Sequence

http://bit.ly/3L5kfwV

"You don't get good at coding by watching, you get good by practising." – Sister Teresa

2.2.1 Exam Questions on Programming Fundamentals

Question	Tip to answer (student)
1. Which statement is an example of **output**? (Tick one) ☐ input("Name?") ☐ print("Hello") ☐ age = 14 ☐ if age > 10 then (1) print("Hello") [1]	Output = shows something.
2. Which statement is an example of **input**? (Tick one) ☐ print("Enter age") ☐ age = input("Enter age") ☐ age = 12 ☐ age > 10 (1) age = input("Enter age") [1]	Input = user types.
3. State what is meant by a **variable**. (1) A named memory location that stores a value. [1]	"Stores data" in memory.
4. Explain why programs use **variables**. (2) Variables store data while the program runs.[1] The stored value can be used again or changed later.[2]	Store + reuse/change.
5. State what is meant by a **constant**. (1) A named value that cannot change while the program runs. [1]	Like a variable, but fixed.
6. Give one reason why a programmer might use a **constant**. (2) It prevents accidental changes to a fixed value (e.g. VAT).[1] If the value changes in future, you update it once.[2]	"Same value used lots" / easy update.
7. Which statement shows **assignment**? (Tick one) ☐ age == 10 ☐ age = 10 ☐ age >= 10 ☐ age != 10 (1) age = 10 [1]	Assignment uses one equals.
8. Explain the difference between **=** and **==** in many languages. (2) = stores a value in a variable (assignment).[1] == checks if two values are equal (comparison).[2]	Assign vs compare.
9. A program asks for a number and then prints double the number. Write an algorithm using input and output. (3) num = input("Enter a number")[1] answer = num * 2[2] print(answer)[3]	Input → calculate → print.
10. Identify the programming construct used: "instructions run one after another in order". (1)	That's one at a time.

Question	Tip to answer (student)
11. Identify the programming construct used: "the program chooses between two paths based on a condition". (1) Selection [1]	That's a decision.
12. Identify the programming construct used: "the program repeats a block of code". (1) Iteration [1]	That's repeating.

13. A program checks if a user is 13 or over and replies TRUE or FALSE. Complete the missing parts. (3)
`age = input("Age?")`
`if age ____ 13 then`
`  print("_____")`
`Else`
`  print("_____")`
`endif`
>=[1] True.[2] False
	Use >= or >?

14. A student writes: `if age >= 13 AND <= 18` Explain what is wrong and correct it. (3) The second condition is not a full comparison.[1] The variable must be written again.[2] Correct: age >= 13 AND age <= 18.[3]	Must repeat the variable.
15. Describe the difference between a **count-controlled** loop and a **condition-controlled** loop. (4) A count-controlled loop repeats a set number of times (e.g. FOR).[1] It uses a counter.[2] A condition-controlled loop repeats while/until a condition is met (e.g. WHILE / REPEAT UNTIL).[3] The condition decides when the loop stops.[4]	2 points each.
16. A loop runs while `x < 5`. x starts at 0 and increases by 1 each time. How many times will "OK" print? (2) It prints 5 times.[1] Values of x are 0,1,2,3,4.[2]	Count 0,1,2,3,4.
17. Complete the trace table for the algorithm. (4) \n\n01 total = 0 \n02 for i = 1 to 3 \n03 total = total + i \n04 next i \n05 print(total) total changes: 0 → 1 → 3 → 6[1] i values: 1,2,3[2] Output is 6[3] No extra changes.[4]	Track i and total.
18. Write an algorithm that keeps asking for a password until the user types `"letmein"`. (6) pass = input("Password?")[1] while pass != "letmein"[2] print("Try again")[3] pass = input("Password?")[4] endwhile[5] print("Access granted")[6]	Needs loop + input + compare.
19. Explain one use of the operator **MOD** in programs. (2) MOD gives the remainder after division.[1] It can check if a number is even (num MOD 2 == 0).[2]	Remainder / even-odd.

02 KEY VOCABULARY / 2.2.1

Programming fundamentals

Use the correct technical wording before attempting longer answers.

Sequence

Instructions carried out in a specified order.

Selection

A condition chooses which block executes.

Iteration

A block of code executes repeatedly.

Variable

A named storage location whose value may change.

Comparison

An operator such as >= that produces a Boolean result.

03 WORKED EXAMPLE / 2.2.1

Programming fundamentals

Follow a complete example. Notice the method before attempting the separate questions.

Task
Use selection inside iteration to report marks.

Think it through — small steps

1. A list of marks is processed one item at a time.
2. Each mark is compared with the pass threshold of 50.
3. A mark below 50 prints Retry; a mark of 50 or more prints Pass.
4. For marks 35, 50, 67 the output is Retry, Pass, Pass.

Code / trace example

```
for mark in [35, 50, 67]:
    if mark >= 50:
        print("Pass")
    else:
        print("Retry")
```

Why this earns marks
The comparison correctly includes the boundary mark of 50 and shows the correct branch.

SPEC 2.2.1 | SKILL 10 OF 28

Sequence and input/output

REMEMBER

Sequence executes instructions in order; INPUT gets data and OUTPUT displays a result.

THINK OF IT LIKE...

A queue of instructions, performed from top to bottom.

WORKED EXAMPLE / APPLY

INPUT name; OUTPUT "Hello " + name.

YOU DO - EXAM PRACTICE

Write two pseudocode statements to input a user's name and display it. (2)

Write your answer before opening the separate answer section.

SPEC 2.2.1 | SKILL 11 OF 28

Selection with IF

REMEMBER

Selection runs different instructions depending on whether a condition is true.

THINK OF IT LIKE...

A fork in the road controlled by a question.

WORKED EXAMPLE / APPLY

IF mark >= 50 THEN OUTPUT "Pass" ELSE OUTPUT "Try again" ENDIF.

YOU DO - EXAM PRACTICE

A pass mark is 40. Write a suitable IF condition to display "Pass" for 40 or more. (2)

Write your answer before opening the separate answer section.

SPEC 2.2.1 | SKILL 12 OF 28

FOR and WHILE iteration

REMEMBER

Iteration repeats instructions. FOR often counts repetitions; WHILE repeats while a condition stays true.

THINK OF IT LIKE...

A repeating checklist, with or without a known count.

WORKED EXAMPLE / APPLY

FOR i = 1 TO 3: OUTPUT i. Displays 1, 2, 3.

YOU DO - EXAM PRACTICE

Explain one difference between a count-controlled FOR loop and a WHILE loop. (2)

Write your answer before opening the separate answer section.

SPEC 2.2.1 | SKILL 13 OF 28

Arithmetic and comparisons

REMEMBER

Arithmetic operators calculate values; comparison operators produce Boolean conditions.

THINK OF IT LIKE...

A calculator for numbers and a referee for comparisons.

WORKED EXAMPLE / APPLY

7 MOD 3 = 1; 7 DIV 3 = 2; 7 > 3 is TRUE.

YOU DO - EXAM PRACTICE

Evaluate 11 DIV 4 and 11 MOD 4. (2)

Write your answer before opening the separate answer section.

04 INDEPENDENT EXAM PRACTICE / 2.2.1

Programming fundamentals

Do this without viewing the next answer pages. Use a whiteboard first for code/traces.

EXAM-STYLE CHALLENGE · 6 MARKS

A program uses the code below. Write its three outputs in order (three marks). State the comparison change needed to raise the pass threshold to 60 (one mark). Explain what happens to the mark 50 afterwards (two marks).

```
for score in [45, 70, 50]:
    if score >= 50:
        print("Pass")
    else:
        print("Resit")
```

Your response

PREDICT → WRITE → TEST → IMPROVE | Self-mark after finishing the entire topic.

MARK SCHEME | SKILL 10 OF 28

Sequence and input/output

QUESTION

Write two pseudocode statements to input a user's name and display it. (2)

MODEL ANSWER

INPUT name (1). OUTPUT name (1). Equivalent clear pseudocode is acceptable.

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

MARK SCHEME | SKILL 11 OF 28

Selection with IF

QUESTION

A pass mark is 40. Write a suitable IF condition to display "Pass" for 40 or more. (2)

MODEL ANSWER

IF mark >= 40 THEN (1) OUTPUT "Pass" (1). A correctly structured equivalent earns credit.

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

MARK SCHEME | SKILL 12 OF 28

FOR and WHILE iteration

QUESTION

Explain one difference between a count-controlled FOR loop and a WHILE loop. (2)

MODEL ANSWER

A FOR loop commonly uses a counter to control a known number of repetitions (1). A WHILE loop continues according to a Boolean condition, which may give an unknown number of repetitions (1).

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

MARK SCHEME | SKILL 13 OF 28

Arithmetic and comparisons

QUESTION

Evaluate 11 DIV 4 and 11 MOD 4. (2)

MODEL ANSWER

11 DIV 4 = 2 (1). 11 MOD 4 = 3 (1).

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

05 ANSWERS & MARK SCHEME / 2.2.1

Programming fundamentals

This answer belongs to the exam-style challenge on the previous practice page.

6-mark challenge · Model answer
Trace and amend selection

Mark 1 First output: Resit.

Mark 2 Second output: Pass.

Mark 3 Third output: Pass.

Mark 4 Change the comparison to score >= 60.

Mark 5 A score of 50 then fails the new condition.

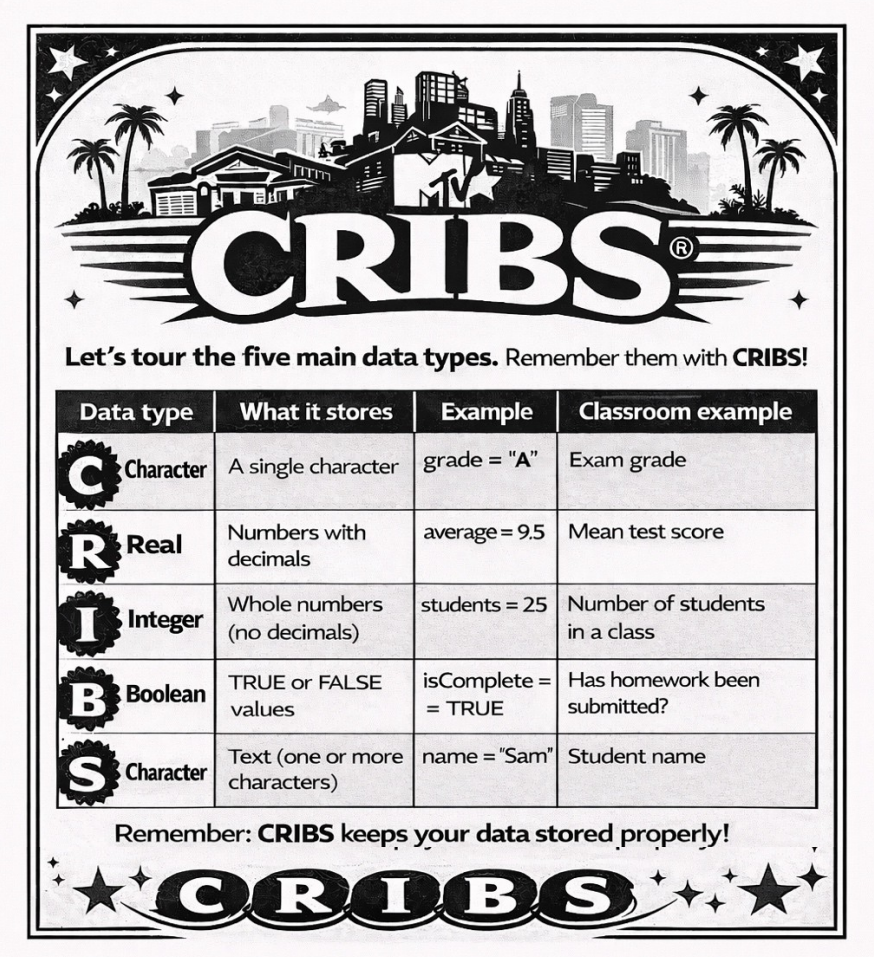
Mark 6 The program prints Resit for that value.

Accept equivalent correct methods, code or relevant wording. Award each point once.

2.2.2 Data Types

2.2.2 Data types	
☐ The use of data types: ○ Integer ○ Real ○ Boolean ○ Character and string ○ Casting	**Required** ✓ Practical use of the data types in a high-level language within the classroom ✓ Ability to choose suitable data types for data in a given scenario ✓ Understand that data types may be temporarily changed through casting, and where this may be useful

A **data type** tells the computer **what kind of data** is being stored and **how it can be used**.



Scenario	Best data type	Why	Common misconception
Number of students	Integer	You can't have part of a student	Using **Real** because "numbers can have decimals"
Test average	Real	Can include decimals	Using **Integer** and losing the decimal part
Homework handed in	Boolean	Only TRUE or FALSE	Using **String** like `"yes"` / `"no"`
GCSE grade	Character	Single letter	Using **String** instead of a single character
Student name	String	Text	Using **Character**, which only stores one letter

Casting means **temporarily changing** a value from one data type to another.
This is often needed because `input` usually returns a s**tring** nd maths needs **numbers**

Casting	Example	Why it's needed
String → Integer	`age = int(age)`	To do maths with input
Integer → String	`print("Score: " + str(score))`	To display numbers with text
Integer → Real	`average = real(total) / real(count)`	To avoid whole-number division

If the question asks you to **choose a data type**, explain **why your choice fits the data** — that's where the marks are.

A school computer system stores the following information: the number of students in a class
the average test score for the class, whether homework has been submitted and a student's exam grade
For each item, **state a suitable data type** and **justify your choice**. (4 marks)

Model answer (4 marks)

- The number of students should be stored as an **Integer** because it is a whole number and cannot have decimals.[1]
- The average test score should be stored as a **Real** because it may include decimal values.[2]
- Whether homework has been submitted should be stored as a **Boolean** because it only has two possible values: TRUE or FALSE.[3]
- A student's exam grade should be stored as a **Character** because it is a single letter such as A or B.[4]

Misconception	Why students think this	Why it is wrong
Using **Real** instead of **Integer**	"All numbers should be Real."	Whole counts (e.g. number of students) cannot have decimal values.
Using **String** instead of **Boolean**	Using `"yes"` / `"no"` feels easier to understand.	Boolean is designed for two-state answers (TRUE / FALSE) and is clearer and more efficient.
Using **String** instead of **Character**	GCSE grades are text, so must be a String.	A single letter (e.g. A) is best stored as a Character.
Using **Integer** for averages	Averages look like normal numbers.	Averages often include decimal values, so must be stored as Real.
Forgetting to cast after input	Input can be used directly in calculations.	`inpu

2.2.2 Exam Questions on Data Types

Question	Tip to answer (student)
1. Which data type is best for storing **True/False**? (Tick one) (1) ☐ Integer ☐ Boolean ☐ Character ☐ String Boolean [1]	True/False = Boolean.
2. State what is meant by a **data type**. (1) A data type describes what kind of value is stored (e.g. integer, string). [1]	"Type of data" stored.
3. Give the most suitable data type for a person's **age**. (1) Integer [1]	Age is a whole number.
4. Give the most suitable data type for a person's **first name**. (1) String [1]	Names are text.
5. Give the most suitable data type for **grade = 'A'**. (1) Character [1]	Single letter.
6. Give the most suitable data type for **height = 1.72**. (1) Real / Float [1]	Has decimals.
7. Explain why a program must know the data type of a variable. (2) It helps the computer store the data correctly (correct memory size/format).[1] It also allows the computer to process it correctly (e.g. maths on numbers, not strings).[2]	Storage size + what you can do with it.
8. A student stores `"2"` in a variable. Explain why `"2" + "2"` gives `"22"` in some languages. (2) The values are strings, not numbers.[1] The + joins text together, so "2" + "2" becomes "22".[2]	String concatenation.
9. A program asks for an age using input(). Explain why the age may need **casting** before doing calculations. (2) input() usually returns text (a string).[1] Casting changes it to an integer so maths can be done correctly.[2]	Input often comes in as a string.
10. State what **casting** means. (1) Converting data from one type to another (e.g. string to integer). [1]	Converting types.
11. A variable `age` currently stores the string `"14"`. Write one line of code/pseudocode to cast it to an integer. (1) age = int(age) [1]	Use int().
12. A variable `score` stores the integer `7`. Write one line of code/pseudocode to cast it to a string. (1) score = str(score) [1]	Use str().
13. Tick (✓) the correct statement: An integer is… (1) ☐ a whole number ☐ text ☐ one character ☐ True/False a whole number [1]	Whole number.
14. Tick (✓) the correct statement: A character is… (1) ☐ multiple words	Single character only.

02 KEY VOCABULARY / 2.2.2

Data types and casting

Use the correct technical wording before attempting longer answers.

Integer

A whole-number value such as 16.

Real / float

A number that may include a fractional part.

String

A sequence of characters, including digits stored as text.

Boolean

A logical value of True or False.

Casting

Converting a value from one compatible type to another.

03 WORKED EXAMPLE / 2.2.2

Data types and casting

Follow a complete example. Notice the method before attempting the separate questions.

Task
Convert user input before a numerical comparison.

Think it through — small steps

1. The text "16" is a string, even though it contains digits.
2. Convert it to an integer before comparing it with 16.
3. The integer 16 meets the >= 16 condition.
4. If the input is not numeric, handle the error rather than assuming the cast will work.

Code / trace example

```
age_text = "16"
age = int(age_text)
if age >= 16:
    print("Eligible")
```

Why this earns marks
The converted value has the appropriate data type for a numerical comparison.

SPEC 2.2.2 | SKILL 14 OF 28

Data types and casting

REMEMBER

A data type defines the kind of value stored. Casting converts a value between compatible types.

THINK OF IT LIKE...

Use the right container for the type of data.

WORKED EXAMPLE / APPLY

score = INTEGER("42") converts text into an integer, assuming the text is numeric.

YOU DO - EXAM PRACTICE

Choose a suitable data type for a student's age and for whether they are present. (2)

Write your answer before opening the separate answer section.

04 INDEPENDENT EXAM PRACTICE / 2.2.2

Data types and casting

Do this without viewing the next answer pages. Use a whiteboard first for code/traces.

EXAM-STYLE CHALLENGE · 6 MARKS

An app receives the input "18" as text. Explain why casting is needed before adding 2. State the resulting integer value. Then describe a possible fault if the user enters "hello", one defensive check, and what the program should do if the check fails.

Your response

PREDICT → WRITE → TEST → IMPROVE | Self-mark after finishing the entire topic.

MARK SCHEME | SKILL 14 OF 28

Data types and casting

QUESTION

Choose a suitable data type for a student's age and for whether they are present. (2)

MODEL ANSWER

Age: integer (1). Present: Boolean (1).

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

05 ANSWERS & MARK SCHEME / 2.2.2

Data types and casting

This answer belongs to the exam-style challenge on the previous practice page.

6-mark challenge · Model answer
Casting and validation

Mark 1 The value "18" is initially a string, not a number.

Mark 2 Convert/cast it to an integer before numerical addition.

Mark 3 18 + 2 = 20.

Mark 4 Trying to convert "hello" to an integer fails because the text is non-numeric.

Mark 5 Validate that the input contains an acceptable integer before attempting the conversion.

Mark 6 If it is invalid, request another entry or handle the conversion error safely.

Accept equivalent correct methods, code or relevant wording. Award each point once.

Question	Tip to answer (student)
☐ one symbol/letter/digit ☐ a decimal number ☐ True/False one symbol/letter/digit [1]	
15. Explain the difference between a **string** and a **character**. (2) A character is one single symbol (e.g. 'A').[1] A string is a sequence of characters (e.g. "Amelia").[2]	One vs many.
16. A program stores whether a user is logged in. Give a suitable variable name and data type. (2) Example: isLoggedIn[1] as Boolean[2]	Name + Boolean.
17. A student writes: `total = "10" + 5` and gets an error. Explain why and show one fix using casting. (3) "10" is a string but 5 is an integer, so they can't be combined as numbers/text safely.[1] Cast "10" to int: total = int("10") + 5[2] (or cast 5 to string for joining text).[3]	Mixed types. Fix by casting string→int or 5→string.
18. A shop stores prices like 2.50 and 19.99. Explain why **integer** is not suitable and give the correct type. (2) Prices can include decimal values, so integers would lose the fractional part.[1] Use Real/Float.[2]	Needs decimals.
19. Complete the table by choosing the most appropriate data type. (4)	Think: text / whole / boolean / char.
20. A program asks for two numbers as input and outputs their total. Complete the missing lines. (4) \n\nnum1 = input("Enter number 1") \nnum2 = input("Enter number 2") \nnum1 = _____(num1) \nnum2 = _____(num2) \nprint(num1 + num2) int[1] int[2] Correct casting of both inputs[3] Correct total output as number[4]	Need int() both times.
Explain one reason why Boolean only needs **1 bit**. (2) Boolean has only two values (True/False).[1] Two states can be stored using one binary digit (0/1).[2]	Only two states.

Variable	Data Type
Username	String
Number of Lives	Integer
Game Over (True/False)	Boolean
Menu Choice (Single Letter)	Character

2.2.3 Additional Programming Techniques

2.2.3 Additional programming techniques	
☐ The use of basic string manipulation ☐ The use of basic file handling operations: ○ Open ○ Read ○ Write ○ Close ☐ The use of records to store data ☐ The use of SQL to search for data ☐ The use of arrays (or equivalent) when solving problems, including both one-dimensional (1D) and two-dimensional arrays (2D) ☐ How to use sub programs (functions and procedures) to produce structured code ☐ Random number generation	**Required** ✓ Practical use of the additional programming techniques in a high-level language within the classroom ✓ Ability to manipulate strings, including: ▪ Concatenation ▪ Slicing ✓ Arrays as fixed length or static structures ✓ Use of 2D arrays to emulate database tables of a collection of fields, and records ✓ The use of functions ✓ The use of procedures ✓ Where to use functions and procedures effectively ✓ The use of the following within functions and procedures: ▪ local variables/constants ▪ global variables/constants ▪ arrays (passing and returning) ✓ SQL commands: ▪ SELECT ▪ FROM ▪ WHERE ✓ Be able to create and use random numbers in a program

2.2.3 String Handling

A **string** is text made up of characters, for example message = “You are amazing!”. String manipulation is **changing or working with text in a program**, such as joining, shortening, or changing letters.

message	Y	o	u	(space)	a	r	e	(space)	a	m	a	z	i	n	g	!
index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Indexing is how a computer counts positions in a string.
The computer starts counting from 0, not 1.Using the message: `"You are amazing!"`
The letter Y is at index 0
The letter o is at index 1
The letter u is at index 2
The space after *You* is at index 3
The last character ! is at index 15
So when we say index, we mean the position of a character in the text, starting from 0.

Feature	What it does	Example code	Result
String	Stores text	subject = "ComputerScience"	`"ComputerScience"`
Length `.length`	Counts characters in a string	subject.length	`15`
Substring `.substring(x, i)`	Takes part of a string (from index x, i characters)	subject.substring(3, 5)	`"puter"`
Left `.left(i)`	Takes first i characters	subject.left(4)	`"Comp"`
Right `.right(i)`	Takes last i characters	subject.right(3)	`"nce"`
Concatenation +	Joins strings together	"Hello " + name	`"Hello Sam"`
Uppercase `.upper`	Converts to uppercase	subject.upper	`"COMPUTERSCIENCE"`
Lowercase `.lower`	Converts to lowercase	subject.lower	`"computerscience"`
ASCII `ASC()`	Converts character to number (	ASC('A') ASC('B') ASC('C')	65 66 67
Character `CHR()`	Converts number to character	CHR(97) CHR(98) CHR(99)	'a' 'b' 'c'

2.2.3 One Dimensional Arrays

Strings are sequences of characters and work like one-dimensional arrays, which helps programmers understand indexing. One-dimensional arrays store lists of data under one variable name. A 1D array could contain **c**haracters, ***r***eals, **i**nts, **b**ooleans or **s**trings.

Each item in the 1D array is referenced by an index, starting at **zero**.

1D Array Question Scores = [65, 72, 81, 90, 58]	Scores = [65, 72, 81, 90, 58]
Output the first score.	Print (Scores[0])
Output the last score in the array.	Print Scores([4])
Output all values in the array (one per line).	FOR i = 0 TO 4 Print (Scores[i]) NEXT i
Find the total (sum) of the scores in the array.	Total =0 FOR i = 0 TO 4 Total =Total + Scores[i] NEXT i Print (Total)
Find the average of the 5 scores in the array.	Total = 0 FOR i = 0 TO 4 Total = Total + Scores[i] NEXT i Average = Total / 5 Print (Average)
Count how many values in the array are greater than 50.	Count = FOR i <- 0 TO 5 IF Scores[i] > 50 THEN Count <- Count + 1 ENDIF NEXT i Print (Count)
Find the highest number in the array.	Max = Numbers[0] FOR i <- 1 TO 5 IF Numbers[i] > Max THEN Max = Numbers[i] ENDIF NEXT i Print (Max)

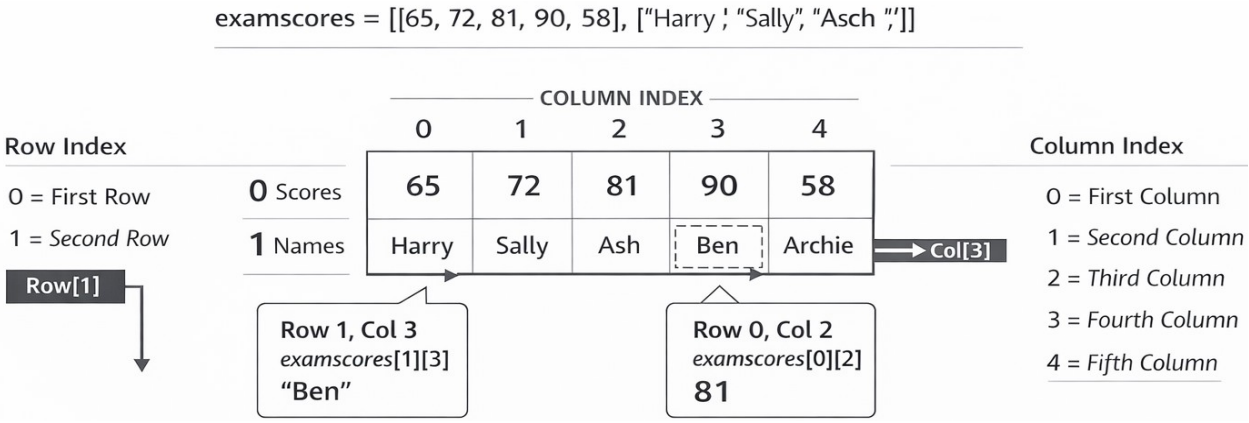
Swap two elements (swap index 1 and index 3), then output the array.	Temp = A[1]
	A[1] = A[3]
	A[3] = Temp //Swapped them – Think bubble!
	FOR i = 0 TO 4
	  Print A[i]
	NEXT i

Replace every value in the array with itself doubled, then output the array.	FOR i = 0 TO 4 Numbers[i] =Numbers[i] * 2 NEXT i

	Print Numbers[i] NEXT i

2.2.3 Two Dimensional Arrays

Two-dimensional arrays store data in rows and columns like a table.



2D Array Question (Tic-Tac-Toe)	Code
2D Array Setup (empty board)	board = [["", "", ""], ["", "", ""], ["", "", ""]]
Put an **X** in the centre square. · · · · X · · · ·	board[1][1] = "X"
Put an **O** in the top-left square. O · · · · · · · ·	board[0][0] = "O"
Print the value in the bottom-right square. O · · · O · · · X	print(board[2][2])
Print the whole board (all 9 squares), one per line.	FOR r = 0 TO 2 FOR c = 0 TO 2 print(board[r][c]) NEXT c NEXT r
Check if **X** has won across the **middle row**. Print TRUE/FALSE.	IF board[1][0] = "X" AND board[1][1] = "X" AND board[1][2] = "X" THEN print(TRUE) ELSE print(FALSE)

Check if **O** has won down the **first column**. Print TRUE/FALSE.	IF board[0][0] = "O" AND board[1][0] = "O" AND board[2][0] = "O" THEN print(TRUE) ELSE print(FALSE) ENDIF

2.2.3 File Handling

File handling is **opening, reading from, writing to, and closing files** so a program can save and use data

Action	What it does (simple)	Example code	Common mistakes
Open file	Opens a file so it can be used	myFile = open("sample.txt")	Forgetting to store the file in a variable
Read line	Reads the next line from the file	myFile.readLine()	Expecting it to read the whole file at once
Write line	Writes a new line to the **end** of the file	myFile.writeLine("Add new line")	Thinking it overwrites the file instead of adding
End of file check	Checks if there are no lines left to read	myFile.endOfFile()	Forgetting `NOT` in the while condition
Read whole file	Reads every line until the end	while NOT myFile.endOfFile() print(myFile.readLine())endwhile	Calling `readLine()` twice inside the loop
Close file	Closes the file when finished	myFile.close()	Forgetting to close the file after use
Create new file	Creates a new text file	newFile("myText.txt")	Forgetting the file must still be opened before writing

4 Mark, typical question

A text file is used to store pilot's codes and date of birth.

The text file is named pilots.txt. The contents of some of this file is shown

```
JP670,17/05/1986
CC200,31/06/1980
SM720,01/04/1976
GC730,06/06/1978
```

Design an algorithm to:

- Ask the user to input a pilot code and a date of birth
- Write these inputs to the pilots.txt text file

1 mark Open pilots.txt

1 mark Ask the user for two **inputs** and **store in variables**

```
1 dronepilotData = open("pilots.txt")
2 pilotCode = input("enter code")
3 dob = input("enter date of birth")
4 dronepilotData.writeLine(pilotCode, dob)
5 dronepilotData.close()
```

1 mark Write both inputs to opened text file

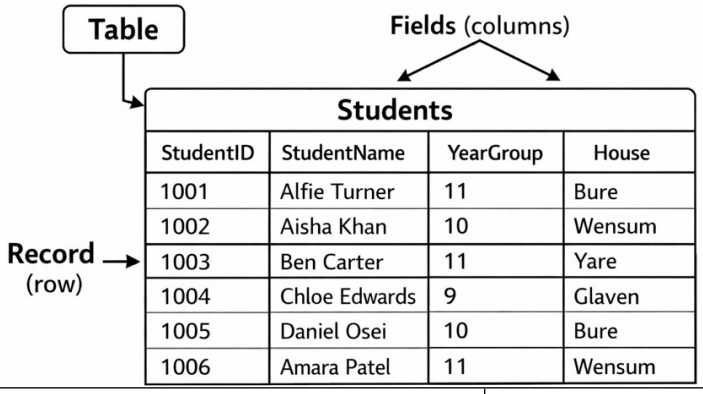
1 mark Close text file

SQL is used to **query a database** (ask questions and get data back) from tables.
SELECT – chooses which **fields (columns)** you want to see in the results.
FROM – tells the database which **table** the data comes from.
WHERE – filters the **records (rows)** so only those that meet a condition are shown.

- **=** means **equal to**, so records with that value are included
- **<>** means **not equal to**, so records with that value are excluded.
- **AND** makes sure **all conditions** must be true for a record to be returned.

> means greater than, so a numeric value must exceed the limit.

- **LIKE** – searches for text that **matches a pattern**, such as names starting with a letter.
- ***** – means **all fields** in the table



Students

StudentID	StudentName	YearGroup	House
1001	Alfie Turner	11	Bure
1002	Aisha Khan	10	Wensum
1003	Ben Carter	11	Yare
1004	Chloe Edwards	9	Glaven
1005	Daniel Osei	10	Bure
1006	Amara Patel	11	Wensum

SQL query	Records returned
SELECT * FROM Students;	1001, Alfie Turner, 11, Bure 1002, Aisha Khan, 10, Wensum 1003, Ben Carter, 11, Yare 1004, Chloe Edwards, 9, Glaven 1005, Daniel Osei, 10, Bure 1006, Amara Patel, 11, Wensum
SELECT StudentName, House FROM Students;	Alfie Turner, Bure Aisha Khan, Wensum Ben Carter, Yare Chloe Edwards, Glaven Daniel Osei, Bure Amara Patel, Wensum
SELECT StudentID, StudentName FROM Students WHERE YearGroup = 11;	1001, Alfie Turner 1003, Ben Carter 1006, Amara Patel
SELECT StudentName, YearGroup FROM Students WHERE House = 'Bure';	Alfie Turner, 11 Daniel Osei, 10
SELECT StudentID, StudentName FROM Students WHERE YearGroup = 10 AND House = 'Bure';	1005, Daniel Osei
SELECT StudentName, House FROM Students WHERE House = 'Wensum' OR House = 'Yare';	Aisha Khan, Wensum Ben Carter, Yare Amara Patel, Wensum
SELECT StudentID, StudentName, House FROM Students WHERE House <> 'Bure';	1002, Aisha Khan, Wensum 1003, Ben Carter, Yare 1004, Chloe Edwards, Glaven 1006, Amara Patel, 11, Wensum
SELECT StudentID, StudentName FROM Students WHERE StudentName LIKE 'A%';	1001, Alfie Turner 1002, Aisha Khan 1006, Amara Patel

Write SQL (3 mark question)

The log is stored in a database table called `events`. The current contents of `events` is shown:.

Date	SensorID	SensorType	Length
05/02/2023	WS2	Window	38
05/02/2023	MS1	Motion	2
06/02/2023	DS3	Door	1
06/02/2023	MS2	Motion	3
06/02/2023	MS1	Motion	2
07/02/2023	WS1	Window	24
07/02/2023	DS1	Door	1

Write an SQL statement to display the sensor IDs of the door sensors that have been triggered for more than 20 seconds.

1 mark SELECT SensorID or you could use WILDCARD **SELECT ***

1 mark **FROM** events

1 mark **WHERE** Length > 20 **AND** sensorType = "Door"

This would return no records as all Doors are open for one second. Well done OCR!

2.2.3 2D arrays to emulate SQL

Let's write the SQL to output all windows that are open for >30 seconds. Include the Date and SensorID.

SQL	Code
SELECT Date, SensorID FROM events WHERE SensorType = 'Window' AND Length > 30; This would return 05/02/2023 Window	eventfile = open("events.csv") line = readline() // skip header row while NOT eventfile.endOfFile() line = readline() Date = parts[0] SensorID = parts[1] SensorType = parts[2] Length = parts[3] if SensorType = "Window" AND Length > 30 then print(Date + " " + SensorID) endif endwhile eventfile.close()

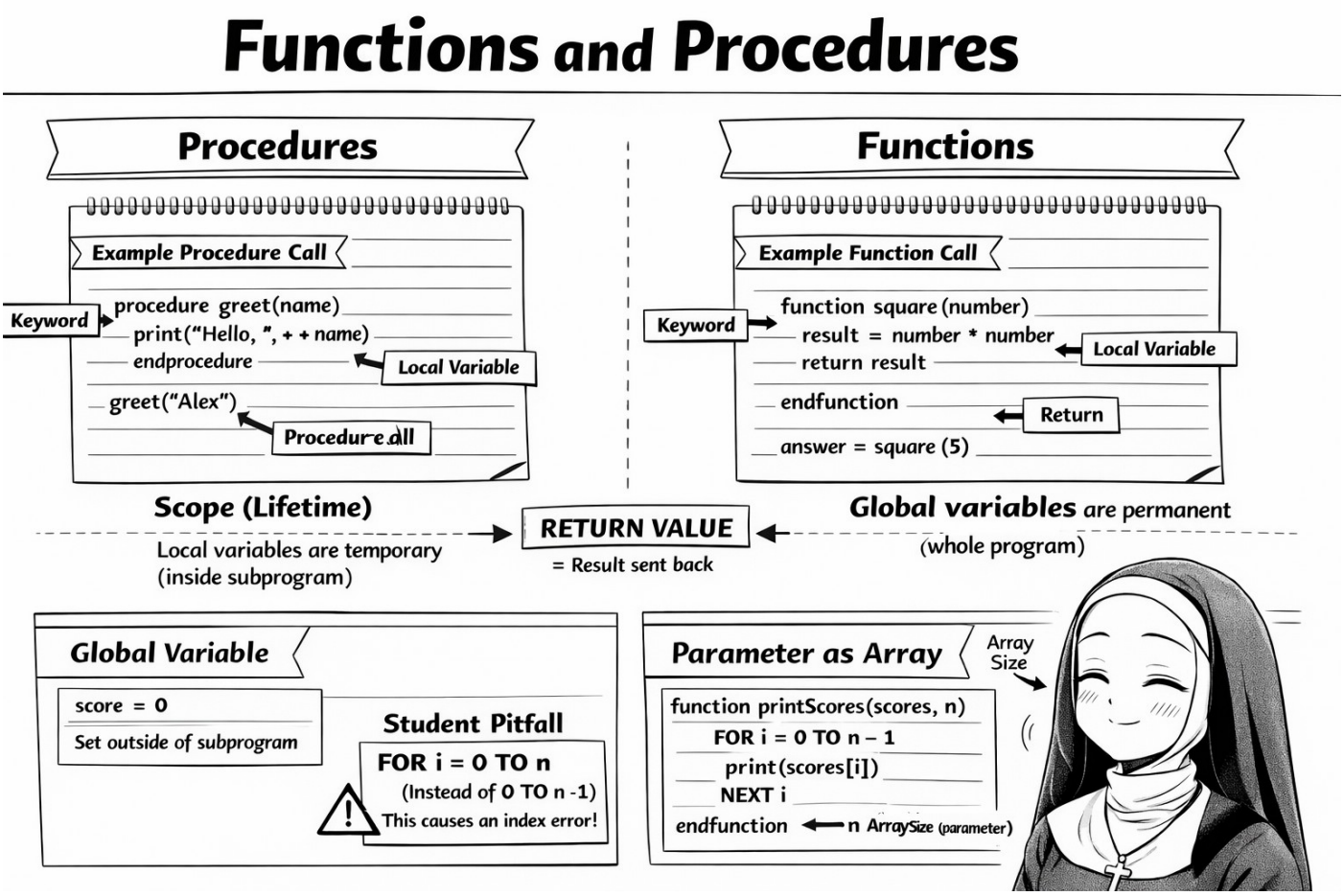
Iterate through records → check field values → print matching records.

2.2.3 Subprograms

A subprogram is a section of code that carries out a task and can be reused. There are two different types of subprograms – procedures and functions. **Procedures** are used to perform tasks such as **displaying output**, while **functions** are used to calculate and **return values** that can be **reused** elsewhere in a program.

👉 **Procedures DO** 👉 **Functions RETURN**

Using subprograms helps: reduce repeated code, make programs easier to read and make programs easier to fix and test (think coding teams).



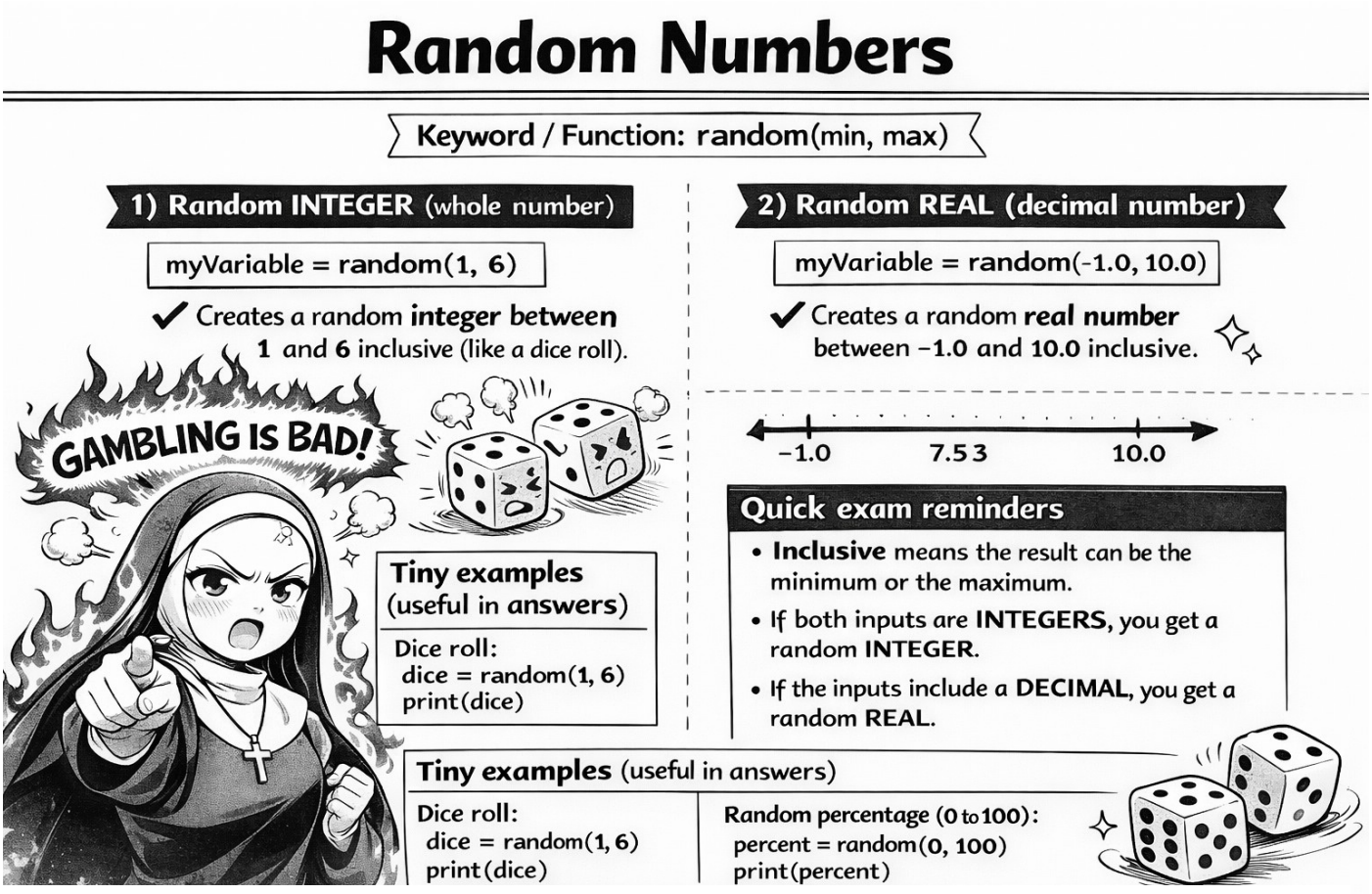
Procedure (code examples + call)	Function (code examples + call)
procedure agePass() print("You are old enough to ride") endprocedure //Main **agePass()**	function squared(number) return number ^2 endfunction //Main **result =squared(4)**
Procedure printName(name) print(name) endprocedure //Main	function add(a, b) return a + b endfunction //Main **total = add(3, 5)**

procedure multiply(num1, num2)
  print(num1 * num2)
endprocedure
multiply(3, 4)
	function multiply(num1, num2)
	return num1 * num2 endfunction
	//Main
	result = multiply(3, 4)

procedure showScore (score)
  print(score)
endprocedure
//Main
currentscore =10
showScore(currentscore)
	function average(Score1, Score2, Score3)
	  return (Score1 + Score2 + Score3) / 3
	endfunction
	//Main
	Score1 =12
	Score2 =14
	Score3 = 16
	avg = average(Score1, Score2, Score3)

procedure printScores(scores, n)
  n = LENGTH(scores)
  FOR i = 0 TO n - 1
  print(scores[i])
  NEXT i
endprocedure
 	
// Main program
scores = [10, 3, 4, 5]
printScores(scores, 4)
	function updateScores(scores, n)
	  n = LENGTH(scores)
	  FOR i = 0 TO n - 1
	  scores[i] = scores[i] + 1
	  NEXT i
	  return scores
	Endfunction
	
	// Main program
	scores = [10, 3, 4, 5]
	scores = updateScores(scores, 4)

2.2.3 Random numbers



Practice writing out code on paper like in the exam – Sister Teresa

Apply functions and selection: Write a function deliveryCost(parcels) which returns parcels * 4. In the main algorithm store deliveryCost(6) in totalCost; if the returned value exceeds 20, output "Large order", otherwise output "Standard order". A function returns a value; the calling code decides what to do with it.

OCR ERL – combining file handling and casting: Open "scores.txt"; while NOT myFile.endOfFile(), read a line into score, convert it using int(score), and print only scores greater than 50. Close the file after the loop. Explain why casting is needed when comparing the input string with a number.

2.2.3 Exam Questions on Additional Programming Techniques

Question	Tip to answer (student)
1(a) What does `resources.length` return for `resources = "ComputerScienceUK"`? **(1)**	Length = number of characters.
`17`	
1(b) What is returned by `resources.left(4)` for `resources = "ComputerScienceUK"`? **(1)**	left(4) = first 4 characters.
`"Comp"`	
1(c) What is returned by `resources.right(3)` for `resources = "ComputerScienceUK"`? **(1)**	right(3) = last 3 characters.
`"eUK"`	
1(d) What does `ASC("B")` return? **(1)**	ASC = letter → number.
`66`	
1(e) What does `CHR(98)` return? **(1)**	CHR = number → character.
`"b"`	
1(f) Which one is a **valid** way to open a file for reading? **(1)** **A** `myFile = open("results.txt")` **B** `open("results.txt") = myFile` **C** `myFile.open("results.txt")` **D** `open myFile "results.txt"`	file handler on the left.
A	
2(a) State two reasons a program should close a file after using it. **(2)**	Two short points. Think "save properly" + "avoid errors".

Question	Tip to answer (student)
Stops data being lost / ensures writes are finished.[1]Prevents errors / frees the file so other programs can use it.[2]	
2(b) State two features of an **array** **(2)**	Fixed size + same data type.
Declared with a fixed length .[1]Stores items of the same data type.[2]	
3(a) Explain what **concatenation** is, using a string example. **(3)**	Say "join strings" + show example.
Concatenation means joining strings together.[1]It uses + to combine them.[2]Example: `print("Hello " + name)` outputs one longer string.[3]	
3(b) A program reads `temp = myFile.readLine()` and then checks `if int(temp) > 10`. Explain why **casting** is used here. **(3)**	"readLine gives text" → convert to number to compare.
readLine() returns a string (text).[1]The program needs a number to compare with 10.[2]int(temp)converts the string to an integer so the comparison works.[3]	
3(c) Explain the difference between a **procedure** and a **function**. **(3)**	Procedure = no return. Function = returns a value.
Both are subroutines (named blocks of code).[1]A procedure runs code but does not return a value.[2]A function returns a value back to the main program.[3]	
4(a) Write an SQL statement to show **ActID and Stage** for acts playing on **Saturday** in `TblMusic(ActID, Stage, Day, SetLength)`. **(4)**	Commas between fields. WHERE not IF. Quotes for Saturday.
`SELECT ActID, Stage`[1]`FROM TblMusic`[2]`WHERE Day = "Saturday"`[34]	
4(b) Write an algorithm that reads every line from `results.txt` and outputs the line **only if it exceeds 10**. Use OCR Exam Reference Language file handling. **(4)**	open → while NOT endOfFile → readLine → IF → print → close

```
myFile = open("results.txt")[1]
while NOT myFile.endOfFile()[2]
  temp = myFile.readLine()[3]
  if int(temp) > 10then[4]
   print(temp)[4]
    endif
endwhile
myFile.close()
```

4(c) An array `scores` stores 5 integers. Write an algorithm to output the **total** of all values in `scores`. **(4)**	total starts at 0, loop through indexes, add, then print.

```
total = 0[1]
for x = 0 to 4[2]
  total = total + scores[x][3]
next x
```

Question	Tip to answer (student)
4(d) Write a **procedure** called `showWelcome()` that outputs `Welcome to Community Arts Night`, then **call** it. **(4)**	Procedure = no return. Define it, then call it.

```
procedure showWelcome()[1]
  print("Welcome to Community Arts Night")[2]
endprocedure[3]
showWelcome()[4]
```

4(e) Write a **function** called `calculatePrice(tickets)` that returns the total cost at **£12 per ticket**. Then call it for 4 tickets and store the result in `totalPrice`. **(4)**	Function must RETURN (not print). Use the parameter.

```
function calculatePrice(tickets)[1]
  total = tickets * 12[2]
  return total[3]
endfunction
totalPrice = calculatePrice(4)[4]
```

6(a) A 2D array stores stage data like a table: `acts = [[1,"Platinum","Saturday",0.5],[2,"Hills","Saturday",1.5]]` Explain how a 2D array can **represent records and fields**. **(6)**	Row = record, column = field. Use an example with indices.

Each inner list is one row, so it represents one record.[1]Each position inside the inner list is one field (ActID, Stage, Day, SetLength).[2]acts[0] gives the first record.[3]acts[0][1] gives the Stage field from the first record ("Platinum").[4]This is like a database table with rows and columns.[5]It lets a program store and process many records in one structure.[6]

6(b) Library lending algorithm: There are **90** books available. A user can borrow **up to 10** books at a time. The program repeatedly asks how many books they want. If enough books remain, output **Books issued** and reduce `books`. Otherwise output **Not enough books**. The program stops when `books` is **0**. Write an algorithm in OCR Exam Reference Language style. **(6)**	Loop while books > 0. Check 1–10. Only subtract when lending succeeds.

```
books = 90
while books > 0[1]
num = int(input("How many books (1 to 10)?"))[2]
if num<1 OR num>10 then[3]
   print("Not accepted")[3]
else
   if books >= num then[4]
      print("Books issued")[5]
      books = books - num[6]
   else
      print("Not enough books")[5]
   endif
endif
endwhile
```

6(c) Write an algorithm that chooses a random whole number from **1 to 6** and outputs `You rolled:` followed by the number. **(6)**	Use `random(1,6)` then output message + value.

```
roll = random(1,6)[1]
if roll >= 1 AND roll <= 6 then[2]
 print("You rolled: " + roll)[3][4]
else
 print("Error")[5]
endif(A roll is produced each time the program runs.)[6]
```

02 KEY VOCABULARY / 2.2.3

Additional programming techniques

Use the correct technical wording before attempting longer answers.

String operations

Combine, measure or extract sequences of characters.

Array

A collection of indexed items of a common type.

Function

A reusable named block that returns a value.

Procedure

A named block that carries out instructions; returning a value is not its main purpose.

File handling

Opening, reading, writing and closing persistent data.

SQL SELECT / FROM / WHERE

Choose columns, identify a table and filter rows.

03 WORKED EXAMPLE / 2.2.3

Additional programming techniques

Follow a complete example. Notice the method before attempting the separate questions.

Task
Use a function to calculate a total from an array.

Think it through — small steps

1. Define a function with a descriptive name and parameter.
2. Start an accumulator at zero.
3. Visit each array item using a loop and add it to the accumulator.
4. Return the result. For [3,5,2], the function returns 10.

Code / trace example

```
def total_cost(prices):
    total = 0
    for price in prices:
        total = total + price
    return total
```

Why this earns marks
The method shows parameter use, traversal, an accumulator and a returned result.

03 WORKED EXAMPLE / 2.2.3

SQL / complete worked model

Follow a complete example. Notice the method before attempting the separate questions.

Task
Write an SQL query with SELECT, FROM and WHERE.

Think it through — small steps

1. Choose the output column: Name.
2. Identify the relation: Students.
3. Add a condition: Year equals 10.
4. Keep the SQL keywords in the correct order.

Code / trace example

```
SELECT Name
FROM Students
WHERE Year = 10;
```

Why this earns marks
SELECT, FROM and WHERE each have a distinct purpose; no Python comment or pseudo-SQL is required.

SPEC 2.2.3 | SKILL 15 OF 28

String handling

REMEMBER

Strings are sequences of characters. Common operations include concatenation, length and substring extraction.

THINK OF IT LIKE...

Letters in a train: join carriages or take a section.

WORKED EXAMPLE / APPLY

"Com" + "puter" produces "Computer"; LENGTH("DOG") is 3.

YOU DO - EXAM PRACTICE

Explain what concatenation does and give one example. (2)

Write your answer before opening the separate answer section.

SPEC 2.2.3 | SKILL 16 OF 28

One-dimensional arrays

REMEMBER

A one-dimensional array stores multiple elements using one index. Many exam pseudocode examples use index 0.

THINK OF IT LIKE...

A row of numbered lockers.

WORKED EXAMPLE / APPLY

scores = [8, 5, 3]; scores[1] is 5 if zero-indexed.

YOU DO - EXAM PRACTICE

A zero-indexed array contains [6, 2, 9]. State the value at index 2 and the number of elements. (2)

Write your answer before opening the separate answer section.

SPEC 2.2.3 | SKILL 17 OF 28

Two-dimensional arrays

REMEMBER

A two-dimensional array uses a pair of indices, commonly row and column.

THINK OF IT LIKE...

A seating plan or spreadsheet grid.

WORKED EXAMPLE / APPLY

grid = [[1, 2], [3, 4]]; grid[1][0] is 3 (zero-indexed).

YOU DO - EXAM PRACTICE

In grid = [[7, 8], [9, 10]], give grid[0][1] and grid[1][1]. (2)

Write your answer before opening the separate answer section.

SPEC 2.2.3 | SKILL 18 OF 28

Procedures, functions and parameters

REMEMBER

Subprograms package reusable instructions. Functions return a value; procedures perform a task and may not return a value.

THINK OF IT LIKE...

A named machine: give it inputs and reuse its operation.

WORKED EXAMPLE / APPLY

FUNCTION double(x): RETURN x * 2. OUTPUT double(4) displays 8.

YOU DO - EXAM PRACTICE

Explain the difference between a parameter and an argument. (2)

Write your answer before opening the separate answer section.

SPEC 2.2.3 | SKILL 19 OF 28

File handling

REMEMBER

Programs open files to read stored data or write new data; files should be closed after use.

THINK OF IT LIKE...

A notebook that survives after the program stops.

WORKED EXAMPLE / APPLY

OPENFILE "scores.txt" FOR READ; READFILE; CLOSEFILE are conceptual stages.

YOU DO - EXAM PRACTICE

State why a program might write to a file and why it should close the file. (2)

Write your answer before opening the separate answer section.

SPEC 2.2.3 | SKILL 20 OF 28

SQL queries

REMEMBER

SELECT chooses columns; FROM identifies the table; WHERE filters rows using conditions.

THINK OF IT LIKE...

Ask a librarian for columns, a shelf and a filter.

WORKED EXAMPLE / APPLY

SELECT Name FROM Students WHERE Year = 11;

YOU DO - EXAM PRACTICE

Write SQL to show Name from a Students table where Year is 10. (3)

Write your answer before opening the separate answer section.

SPEC 2.2.3 | SKILL 21 OF 28

Random numbers

REMEMBER

Random-value functions generate values suitable for simulations, games and testing; clarify the supported inclusive range.

THINK OF IT LIKE...

A virtual die roll.

WORKED EXAMPLE / APPLY

RANDOM(1, 6) is intended as a random integer from 1 through 6 in this example.

YOU DO - EXAM PRACTICE

Give one use for random values and one reason to test their permitted range. (2)

Write your answer before opening the separate answer section.

04 INDEPENDENT EXAM PRACTICE / 2.2.3

Additional programming techniques

Do this without viewing the next answer pages. Use a whiteboard first for code/traces.

EXAM-STYLE CHALLENGE · 6 MARKS

Write a Python function named count_high(scores) that returns how many scores are STRICTLY greater than 60 in a one-dimensional list. For [40, 61, 80, 60], it must return 2.

Your response

PREDICT → WRITE → TEST → IMPROVE | Self-mark after finishing the entire topic.

MARK SCHEME | SKILL 15 OF 28

String handling

QUESTION

Explain what concatenation does and give one example. (2)

MODEL ANSWER

Concatenation joins strings together (1). E.g. "Good" + "bye" produces "Goodbye" (1).

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

MARK SCHEME | SKILL 16 OF 28

One-dimensional arrays

QUESTION

A zero-indexed array contains [6, 2, 9]. State the value at index 2 and the number of elements. (2)

MODEL ANSWER

Value at index 2: 9 (1). Number of elements: 3 (1).

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

MARK SCHEME | SKILL 17 OF 28

Two-dimensional arrays

QUESTION

In grid = [[7, 8], [9, 10]], give grid[0][1] and grid[1][1]. (2)

MODEL ANSWER

grid[0][1] = 8 (1). grid[1][1] = 10 (1).

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

MARK SCHEME | SKILL 18 OF 28

Procedures, functions and parameters

QUESTION

Explain the difference between a parameter and an argument. (2)

MODEL ANSWER

A parameter is the named variable in the subprogram definition (1). An argument is the actual value supplied when the subprogram is called (1).

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

MARK SCHEME | SKILL 19 OF 28

File handling

QUESTION

State why a program might write to a file and why it should close the file. (2)

MODEL ANSWER

Writing to a file allows data to persist between runs (1). Closing releases the file resource and helps ensure buffered data is saved (1).

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

MARK SCHEME | SKILL 20 OF 28

SQL queries

QUESTION

Write SQL to show Name from a Students table where Year is 10. (3)

MODEL ANSWER

SELECT Name (1) FROM Students (1) WHERE Year = 10; (1). SQL keywords are case-insensitive.

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

MARK SCHEME | SKILL 21 OF 28

Random numbers

QUESTION

Give one use for random values and one reason to test their permitted range. (2)

MODEL ANSWER

Random values can simulate a dice roll or select game events (1). Range tests check the values stay within stated limits (1).

SELF-MARK

Give credit for equivalent correct wording. Correct your answer in a different colour, then revisit the study card if a key point was missing.

05 ANSWERS & MARK SCHEME / 2.2.3

Additional programming techniques

This answer belongs to the exam-style challenge on the previous practice page.

6-mark challenge · Model answer
Write a counting function

Mark 1 Define a function count_high with a scores parameter.

Mark 2 Initialise a counter to 0.

Mark 3 Loop over every score in the list.

Mark 4 Compare with > 60 (not >= 60).

Mark 5 Increase the counter when the condition is true.

Mark 6 Return the counter after the loop; the example returns 2.

Accept equivalent correct methods, code or relevant wording. Award each point once.

WORKED SOLUTION / 2.2.3

Additional programming techniques

See a complete method and compare it with each mark-scheme point.

Complete valid Python answer

```
def count_high(scores):
    count = 0
    for score in scores:
        if score > 60:
            count = count + 1
    return count


print(count_high([40,61,80,60]))  # 2
```

Follow the trace
40 is not greater than 60: count stays 0.
61 is greater: count becomes 1.
80 is greater: count becomes 2.
60 is equal, not greater: count stays 2.
Return 2.